

Software Test Plan (STP) - Task Manager Application

Project: Task Manager Application

Version: 1.0

Authors: Team 4 — OM SATYAJYOTI PATTANAYAK; PRACHI GANESH JOSHI; PRIYANSH SONI;
RAJAT KIRAN KOOGGE

Date: 02-10-2026

Status: Draft

1. Introduction

Purpose: This document defines the test plan for Version 1.0 of the Task Manager Application. It establishes the testing objectives, scope, strategy, resources, schedule, responsibilities, security validation activities, test-case management, and requirements traceability needed to verify the implementation against the approved SRS and SAD.

Scope: Testing covers the complete approved functional scope: user registration, authentication, protected access, task creation, task viewing, task editing, task deletion, task assignment, status management, priority and deadline information, search/filtering, and dashboard-based progress tracking. Testing also covers the applicable non-functional and security requirements: performance, security, usability, reliability, maintainability, testability, authentication, authorization, password protection, and server-side input validation.

References: Task Manager Application Software Requirements Specification (SRS), Version 1.0 approved, 02-10-2026; Task Manager Application Software Architecture and Design Specification (SAD), Version 1.0, 02-10-2026; Software Engineering Mini-Project Deliverables Part-1; Requirements Traceability Matrix (RTM) in Appendix C of the SRS.

Definitions: SRS (Software Requirements Specification), SAD (Software Architecture and Design Specification), STP (Software Test Plan), RTM (Requirements Traceability Matrix), API (Application Programming Interface), JWT (JSON Web Token), REST (Representational State Transfer), FR (Functional Requirement), NFR (Non-Functional Requirement), SR (Security Requirement), CRUD (Create, Read, Update, Delete).

2. Test Items

- React.js frontend: registration, login, task listing, task creation, task editing, assignment, status/priority updates, search/filtering, and dashboard/progress views.
- Authentication Module: registration, credential validation, authenticated access, and protection of protected functionality.
- Task Management Module: task creation, retrieval, editing, deletion, assignment, status, priority, deadline, and authorization-related operations.
- Dashboard / Search Module: authorized task search/filtering and task/progress presentation.

- Node.js/Express API: REST endpoints, request validation, authentication, authorization, JSON responses, and HTTP error handling.
- Database: persistent user and task data, including assignment, status, priority, deadline, ownership/authorization relationships, and successful update persistence.
- Integration between React frontend, Node.js/Express backend, and database.
- Security controls and logging/monitoring behaviour described in the SAD.

3. Features to be Tested

Features mapped to SRS requirement IDs and SAD design references:

- FR-01: User registration with unique email and password — Authentication Module; registration API.
- FR-02: User authentication before protected functionality — Authentication Module; Login Sequence; authentication API.
- FR-03: Task creation with title, description, priority, and deadline where applicable — Task Management Module; Create Task Sequence; task API.
- FR-04: View authorized tasks — Task Management Module; GET /api/tasks.
- FR-05: Edit an authorized task and persist accepted changes — Task Management Module; PUT /api/tasks/:id.
- FR-06: Delete an authorized task and prevent unauthorized deletion — Task Management Module; DELETE /api/tasks/:id.
- FR-07: Assign a task to a registered team member — Task Management Module; PUT /api/tasks/:id/assign.
- FR-08: Update task status using a valid status value — Task Management Module; PUT /api/tasks/:id/status.
- FR-09: Specify and modify task priority using a supported value — Task Management Module; task update API.
- FR-10: Search and filter authorized tasks using supported task attributes — Task Management / Dashboard / Search Module; API and filtering behaviour.
- FR-11: Dashboard displays authorized task progress using consistent indicators — Dashboard Module; UX Design.
- NFR-01: Normal representative task-management requests return an application response within 3 seconds in the agreed test environment.
- NFR-02: Protected functionality is inaccessible to unauthenticated users.
- NFR-03: Interface is consistent and provides understandable labels, validation/error feedback, and clear progress indicators.
- NFR-04: Successfully stored user/task information is preserved and accepted updates are not silently lost.
- NFR-05: Major frontend, backend/service, and data-access components remain modular and maintainable.
- NFR-06: Major functional components and interfaces support independent and integration testing.

- SR-01 to SR-04: Authentication, authorization, password protection, and server-side input validation.

4. Features Not to be Tested

- Features outside the approved SRS scope, including notifications, advanced collaboration, richer analytics, activity history, or other future enhancements.
- Production cloud administration and cloud-provider internals.
- Third-party services not included in the implemented application.
- Database engine internals beyond application-level persistence, retrieval, authorization, and update behaviour.
- Dedicated hardware, sensors, controllers, or physical safety functions because the SRS defines no dedicated hardware interface or safety-critical function.
- Code-file and system-test references designated as Part-2 implementation/test references in the SRS RTM.

5. Test Approach / Strategy

Levels:

- Unit testing: module-level frontend and backend/service behaviour, including validation, authentication helpers, task operations, search/filter logic, and dashboard logic where separately testable.
- Integration testing: React frontend ↔ Node.js/Express API ↔ database, including request/response contracts, persistence, authentication/authorization, and task data flow.
- System testing: end-to-end Task Manager functionality across the approved user workflows.
- Acceptance testing: validation against the approved SRS requirements and acceptance conditions.

Types:

- Functional testing: registration, authentication, task CRUD, assignment, status, priority/deadline, search/filtering, and dashboard.
- Regression testing: repeat affected functional and security tests after changes to major modules or defect fixes.
- Performance testing: measure representative normal task-management response time against the NFR-01 3-second target.
- Security testing: authentication, authorization, protected endpoints, server-side input validation, password/sensitive-data handling, and appropriate 401/403 behaviour.
- Usability testing: navigation, understandable labels, forms, validation/error messages, consistent task information, and dashboard progress indicators.
- Reliability testing: verify successful task/user updates remain persisted and are not silently lost.
- Maintainability/testability checks: verify modular component boundaries, defined API interfaces, and ability to exercise major components independently.

Entry Criteria: Approved SRS and SAD available; stable implementation build delivered; test

environment and database available; required test data prepared; APIs/UI available for testing; and blocking environment issues resolved.

Exit Criteria: 100% of planned test cases executed; no critical defects open; all security validation checks completed; applicable acceptance conditions satisfied; NFR-01 performance evidence recorded; and requirement coverage reviewed.

5.1 Security Validation

- SR-01 / TC-02: Verify successful authentication with valid credentials and rejection of invalid credentials; verify unauthenticated users cannot access protected task functionality.
- SR-02 / TC-07 and TC-10: Verify authorization checks prevent users from viewing, modifying, assigning, or deleting task information for which they lack permission; verify appropriate 403 responses.
- SR-03 / TC-10: Verify passwords and authentication credentials are not exposed in API responses or normal application logs and that the selected secure password-handling mechanism is applied.
- SR-04 / TC-03 and TC-10: Verify server-side validation rejects missing, malformed, unexpected, or invalid registration, authentication, task, search, assignment, and update input.
- NFR-02 / TC-02 and TC-10: Verify protected functions deny unauthenticated access.
- Verify protected API communication uses HTTPS when the application is deployed in a secure environment.
- Verify authentication failures return appropriate 401 responses and authorization failures return appropriate 403 responses without exposing sensitive information.
- Verify task retrieval, search/filtering, dashboard data, assignment, editing, and deletion respect authenticated-user authorization relationships.

6. Test Environment

Hardware: Development/test computers capable of running a supported modern web browser, React.js frontend, Node.js/Express.js backend, and the selected database. No dedicated client hardware is required by the SRS.

Software: React.js frontend, Node.js with Express.js backend, configured persistent database, supported modern web browser, and the Task Manager Application test build.

Tools: Browser developer tools, API testing tools such as Postman or equivalent, Git, and project test tools selected during implementation.

Test Data: Dummy user accounts; unique and duplicate email addresses; valid and invalid credentials; authorized and unauthorized user/task relationships; sample tasks with valid and invalid titles/descriptions, priorities, statuses, and deadlines; assignment data; search/filter values; dashboard data covering to-do, in-progress, completed, and overdue conditions where applicable; and malformed/empty input for validation tests.

7. Test Schedule

Milestones:

- Test plan preparation: 02-Oct-2026
- Test case design and requirements/design traceability: 02-Oct-2026
- Test environment setup: After a stable implementation build is available
- Unit and integration test execution: After environment readiness and component availability
- System test execution: After integrated build is stable
- Security and performance validation: During system testing after required APIs and deployment/test configuration are available
- Test execution end: After all planned test cases are completed and required reruns are finished
- UAT and final test summary: After system testing, defect closure, and acceptance review

8. Test Deliverables

- Test Plan (this document)
- Detailed Test Cases and test data
- Test Scripts, where automation is used
- Test Execution Logs and evidence/screenshots
- Defect Reports and retest results
- Requirements Traceability Matrix updates
- Security validation results
- Performance response-time results
- Test Summary Report

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead	RAJAT KIRAN KOOGA	Prepare the test plan, coordinate execution, monitor coverage and status, and maintain test readiness and reporting.
Test Engineer	PRIYANSH SONI	Design and execute test cases, record actual/expected results, collect evidence, and log and retest defects.
Developer	PRACHI GANESH JOSHI	Support defect analysis, implement fixes, provide stable builds, and support regression and integration testing.
Product Owner	OM SATYAJYOTI PATTANAYAK	Review test results and acceptance evidence and approve readiness for the next project stage.

10. Risks and Mitigation

Risk	Mitigation
Delay in stable build delivery	Prepare test cases and test data early, keep implementation interfaces aligned with the SRS/SAD, and request stable builds before execution.
Test environment or database downtime	Maintain a reproducible local test setup, verify required services before execution, and restore services before resuming tests.
Defects in authentication or authorization	Prioritize security validation, isolate blocking security defects, and perform focused regression testing after fixes.

11. Assumptions & Dependencies

- React.js, Node.js/Express.js, and the selected database test environment will be available before execution.
- Test data for registered users, duplicate registrations, credentials, authorized/unauthorized users, tasks, assignments, statuses, priorities, deadlines, search/filter values, and dashboard conditions will be prepared before execution.
- The implemented APIs and UI will follow the requirements and architecture/design defined in the SRS and SAD, including the API endpoints and error semantics documented in the SAD.
- The database will preserve successfully stored user and task information so reliability tests can verify persistence.
- Team members will provide stable builds, required environment access, and defect fixes required for planned testing.
- Part-2 implementation and system-test references will be added to the RTM when the implementation and system tests exist.

12. Suspension & Resumption Criteria

Suspend testing if:

- The test environment, database, API, or application build is unavailable for an extended period or prevents planned execution.
- A blocking defect prevents execution of a substantial portion of the planned test cases or prevents safe/security-valid testing.
- The application build is too unstable to produce meaningful or repeatable test results.
- A major requirement/design change invalidates a substantial portion of the prepared test cases.

Resume testing if:

- Blocking defects are resolved or an approved stable build is provided.
- The test environment and required services are restored.
- Affected test cases are reviewed and updated after an approved requirement/design change.
- Required test data and API/UI interfaces are available and stable.

13. Test Case Management & Traceability

The RTM ensures mapping of every SRS requirement to one or more planned test cases. Test cases are aligned with the SRS Appendix C RTM and the corresponding SAD architecture/design references.

- TC-01 (FR-01): User registration. Preconditions: registration page/API available. Steps: submit valid unique email and password; repeat with missing/invalid input; repeat with an already registered email. Expected: valid registration creates one account; invalid/incomplete input is rejected; duplicate email is not registered twice. SAD trace: Authentication Module and registration API.
- TC-02 (FR-02, SR-01): User authentication and protected access. Steps: log in with valid credentials; attempt invalid credentials; attempt protected task access without authentication. Expected: valid credentials establish authenticated access; invalid credentials are rejected; unauthenticated protected requests are denied with appropriate authentication behaviour. SAD trace: Authentication Module, Login Sequence, authentication API.
- TC-03 (FR-03, SR-04): Create task and validate input. Steps: create a task with valid title, description, priority, and deadline; repeat with missing/invalid required input. Expected: valid task is stored and returned/displayed; invalid input is rejected by server-side validation. SAD trace: Task Management Module, Create Task Sequence, task API.
- TC-04 (FR-04): View authorized tasks. Steps: request the task list as an authenticated user with authorized task data and compare with data outside the user's authorization. Expected: authorized task information is returned/displayed; unauthorized task information is not exposed. SAD trace: Task Management Module, GET /api/tasks, Security Architecture.
- TC-05 (FR-05): Edit an authorized task. Steps: change supported task fields for an authorized task and submit; retrieve the task afterward. Expected: accepted changes are persisted and reflected in subsequent retrieval; unauthorized edit is rejected. SAD trace: Task Management Module, PUT /api/tasks/:id.
- TC-06 (FR-06): Delete an authorized task. Steps: delete an authorized task; attempt unauthorized deletion. Expected: authorized deletion removes the task from subsequent retrieval; unauthorized deletion is rejected with appropriate authorization behaviour. SAD trace: Task Management Module, DELETE /api/tasks/:id.
- TC-07 (FR-07, FR-08, FR-09, SR-02): Assignment, status, and priority. Steps: assign a task to a registered team member; change status to a valid supported value; modify priority to a supported value; attempt unauthorized changes. Expected: each valid change is stored and reflected in task details/list/dashboard; invalid or unauthorized changes are rejected. SAD trace: Task Management Module, assignment/status APIs, Security Architecture.
- TC-08 (FR-10): Search and filtering. Steps: search using supported task attributes and apply filters; include data the current user is not authorized to access. Expected: matching authorized tasks are returned; unauthorized task information is excluded; filtering remains consistent with the authorized task set. SAD trace: Dashboard/Search Module and API design.
- TC-09 (FR-11, NFR-03): Dashboard and usability. Steps: open the dashboard with

representative task data; verify counts/progress indicators; navigate through registration/login/task-management forms; inspect labels, validation feedback, error messages, and task information consistency. Expected: authorized progress information is summarized clearly; navigation and labels are understandable; interface behaviour and feedback are consistent. SAD trace: Dashboard/Search Module and UX Design.

- TC-10 (NFR-01, NFR-02, NFR-04, NFR-05, NFR-06, SR-02, SR-03, SR-04): Performance, security, reliability, maintainability, and testability. Steps: measure representative normal registration/login/task-management API responses; verify 3-second target; attempt unauthorized operations; inspect responses/logs for sensitive credential exposure; submit malformed input; verify successful updates persist; verify major frontend/backend/data interfaces can be exercised independently and through integration tests. Expected: NFR-01 response target is met in the agreed environment; unauthorized actions are rejected; sensitive credentials are not exposed; invalid input is rejected server-side; successful data remains persisted; defined component/API boundaries support independent and integration testing. SAD trace: Architecture Goals & Constraints, Security Architecture, API Design, Error Handling/Logging/Monitoring.

For each test case, the execution record shall capture preconditions, test data, steps, expected result, actual result, pass/fail status, evidence reference, defect reference where applicable, and the corresponding SRS requirement ID. Code-file and system-test references remain Part-2 implementation/test references until those artefacts exist.

14. Test Metrics & Reporting

Metrics collected:

- Percentage of planned test cases executed.
- Percentage of test cases passed, failed, and blocked.
- Requirement coverage across FR-01 to FR-11, NFR-01 to NFR-06, and SR-01 to SR-04.
- Defect count by severity and current status.
- Defect aging and retest status.
- Security validation completion for SR-01 to SR-04 and NFR-02.
- Performance response-time results against the NFR-01 3-second target.
- Reliability/persistence validation results for NFR-04.
- Maintainability/testability verification evidence for NFR-05 and NFR-06.

Reports:

- Test execution status.
- Defect status report.
- Requirements/test traceability status.
- Security validation report/results.
- Performance test results.
- Final Test Summary Report.

15. Approvals

Role	Name	Signature / Date
QA Lead	Rajat Kiran Kooge	Rajat Kiran Kooge /02-10-2026
Dev Lead	Prachi Ganesh Joshi	Prachi Ganesh Joshi /02-10-2026
Product Owner	Om Satyajyoti Pattanayak	Om Satyajyoti Pattanayak/02-10-2026
Test Engineer	Priyansh Soni	Priyansh Soni /02-10-2026